NLP Paper & Chatbot Blueprint: Intent-Routing Classifier for Maersk's Dify Knowledge Base

Executive Summary

This report provides a step-by-step guide for building a knowledge-base-routing chatbot for Maersk using Dify, and structures the accompanying NLP course paper. The core academic contribution is a systematic comparison of intent classification approaches — from Bag-of-Words baselines to fine-tuned BERT — applied to a real commercial ticket dataset, exposing the failure modes of Copilot-generated zero-shot classifiers and replacing them with a ticket-grounded model. This maps directly to every NLP course requirement while producing genuine business value.

Part 1: Paper Structure

The paper should follow an academic NLP structure with six main sections. Below each section heading is an explanation of exactly what to write and why.

1. Introduction (½–1 page)

What to write: Open with the business problem — Maersk's Dify-hosted chatbot routes customer queries to one of N knowledge base sections, but the current classifiers (generated by Copilot from KB descriptions alone) misroute a significant share of tickets. State the research question: *"Can training intent classifiers directly on historical ticket data improve routing accuracy over zero-shot LLM-based classifiers?"*

Why it matters academically: This frames the project as a comparison between human-language-grounded models and KB-structure-grounded models, which is a genuine open question with commercial stakes. Maersk processes millions of customer interactions, and AI-driven support automation already contributes to their operational efficiency. Misrouting has real costs: wrong knowledge base → wrong answer → human escalation.[1][2]

Course requirement fulfilled: Relevance argument.

2. Related Literature (1–1.5 pages)

Review the following four papers and connect each explicitly to your project:

Paper 1 — REIC: RAG-Enhanced Intent Classification at Scale (Zhang et al., Amazon, EMNLP 2025)[3][4]
This is your anchor paper. REIC demonstrates that fine-tuned RoBERTa substantially outperforms zero-shot Claude/GPT for in-domain intent classification at scale. It provides a

direct experimental precedent for your hypothesis that ticket-trained classifiers beat Copilot zero-shot classifiers. REIC also introduces the idea of combining retrieval (from labeled examples) with classification, which you can optionally implement as an advanced variant.

Paper 2 — Benchmarking Conversation Routing in Chatbot Systems (Maksymenko et al., CEUR-WS 2024)[^5]
Benchmarks three routing paradigms: NLI zero-shot (BART-MNLI), LLM in-context (GPT-4o), and embedding-based semantic router. Key empirical finding: label name wording and order in the LLM prompt significantly affect accuracy — this directly explains why Copilot-generated classifiers are fragile. Use this to motivate your embedding-based and fine-tuned BERT approaches as more stable alternatives.

Paper 3 — Improving Automated Categorization of Customer Requests (Mejzlík & Bureš, EJOBSAT 2024)[6][7]
Works on real service-desk ticket datasets. Finds that (a) class imbalance dominates real ticket distributions, (b) fine-tuned transformer features are essential for good performance, and (c) preprocessing quality matters more than model architecture. Use this to frame your data description and preprocessing decisions.

Paper 4 — Enhancing Chatbot Intent Classification using Active Learning Pipeline (Kuligowska & Kowalczuk, 2024)[8][9]
Addresses the labeling bottleneck: your raw ticket dataset is likely unlabeled (tickets exist, but aren't tagged with KB categories). This paper proposes sentence-transformer clustering + Active Learning to label data efficiently without annotating every example. Use this to justify your labeling methodology.

Connecting to your project: End the literature section with a clear gap statement — existing work either studies synthetic datasets (lacking real-world messiness) or uses zero-shot LLMs (lacking ticket-grounded training). Your project fills the gap by applying a full spectrum of classifiers to a real-world logistics ticket corpus.

3. Data Description (1 page)

What to write:

- **Source:** Maersk's internal Dify chatbot ticket dataset. Describe the schema: ticket text (user query), optionally: timestamp, session ID, KB category assigned by current classifier. Note whether ground-truth labels exist or need to be created.
- **Size:** Report total number of tickets, number of unique intents/categories (from the KB), average ticket length in tokens/words.
- **Sample rows:** Include 3–5 anonymized example tickets showing the range of phrasing, e.g., formal ("Please provide tracking status for booking reference X") vs. informal ("where is my stuff??").
- **Class distribution:** Plot or describe the distribution across KB categories — this will almost certainly be imbalanced (shipping tracking likely dominates, while edge categories like dangerous goods handling are rare).[10][6]

- **Knowledge base structure:** Briefly describe the KB — how many documents, what file types (MD, DOCX, PDF, CSV), how categories are defined. Note that the current Dify classifiers were built by Copilot from these KB category names/descriptions alone, without seeing the tickets.

Course requirement fulfilled: Data Description section.

4. ML Task & Methods (2–3 pages)

This is the core technical section. Structure it as a progression from simple to complex, with each method building on the previous.

4.1 Task Definition

Define the task formally: given a ticket text x, predict the KB category label $y \in \{c_1, c_2, \ldots, c_K\}$ where K is the number of KB sections. This is a **multi-class single-label classification** problem. Evaluation metrics: macro-F1 (primary — treats all classes equally regardless of size, crucial given class imbalance), weighted-F1 (secondary), and accuracy.[11][12][13][14]

Use **macro-F1 as your primary metric** because your ticket distribution is imbalanced — accuracy would reward a model that only predicts the most common category. Report all three so results are reproducible.[13][15]

4.2 Preprocessing Pipeline

Apply consistently across all models:

1. Lowercase normalization
2. Remove PII (booking reference numbers, email addresses) to avoid spurious correlations
3. Tokenization (whitespace for BoW; BERT tokenizer for transformer models)
4. For BoW methods: stop-word removal, lemmatization
5. Train/validation/test split: 70/15/15 stratified by class to preserve class ratios

Class imbalance handling: Apply SMOTE (Synthetic Minority Oversampling Technique) to training data for the BoW+ML classifiers. For BERT, use weighted cross-entropy loss instead, since SMOTE on high-dimensional embeddings is less effective.[16][17]

4.3 Baseline Models — Bag-of-Words + Classical ML

These fulfill the BoW, n-gram, and classical ML course requirements.[18][11]

Model 1 — TF-IDF + Naive Bayes

- Representation: TF-IDF unigrams and bigrams (n=1,2), max 10,000 features
- Classifier: Multinomial Naive Bayes (α=1.0)

- Rationale: Extremely fast to train; establishes an interpretable performance floor. Naive Bayes is well-suited to sparse high-dimensional text features[^19]

Model 2 — TF-IDF + Logistic Regression

- Representation: TF-IDF unigrams and bigrams, L2 regularization
- Classifier: Logistic Regression (C=1.0, solver=lbfgs, max_iter=1000)
- Rationale: Consistently strong TF-IDF baseline, often outperforms Naive Bayes on multi-class problems[20][18]

Model 3 — TF-IDF + Random Forest

- Representation: Same TF-IDF features
- Classifier: Random Forest (n_estimators=200)
- Rationale: Non-linear tree-based model; covers the decision tree/ensemble requirement from the course

4.4 Word Embedding Models

Model 4 — Sentence-Transformer Embeddings + Cosine Similarity Router

- Representation: Encode each ticket with `paraphrase-multilingual-MiniLM-L12-v2` (supports Danish/English mixed queries) or `all-MiniLM-L6-v2`
- Routing: Compute centroid embedding for each KB category from labeled training examples, then route new tickets to nearest centroid by cosine similarity
- Rationale: This is the embedding-based semantic router approach shown to nearly match GPT-4o accuracy in the benchmarking paper. It also maps directly to how Dify's knowledge base retrieval works, making integration trivial[21][22][^5]

Model 5 — Sentence-Transformer Embeddings + Logistic Regression

- Same embeddings as Model 4, but use the embedding vectors as features for a Logistic Regression classifier
- Rationale: Tests whether a learned decision boundary on embedding space outperforms nearest-centroid routing

4.5 Pre-trained Language Models

Model 6 — Fine-tuned BERT

- Base model: `bert-base-multilingual-cased` (handles English + Danish/other Maersk ticket languages) or `bert-base-uncased` if tickets are English-only
- Architecture: BERT encoder → [CLS] token → dropout (p=0.1) → linear layer → softmax over K classes
- Training: Fine-tune all layers for 3–5 epochs, batch size 16, learning rate 2e-5, AdamW optimizer with linear warmup, weighted cross-entropy loss for class imbalance[23][24][^25]

- Rationale: Fine-tuned BERT consistently outperforms classical ML on in-domain ticket classification tasks. The [CLS] token representation captures contextual meaning — critical for ambiguous shipping queries[23][6]

Model 7 — GPT Zero-Shot (Baseline for LLM comparison)

- Method: Send each ticket to GPT-4o-mini with a system prompt listing the KB category names and descriptions (mirroring what Copilot generated for the current Dify classifier)
- This is your **reproduction of the current broken classifier** — it gives you an exact empirical baseline to compare against[21][5]

Model 8 (Optional/Advanced) — GPT Few-Shot

- Same as Model 7 but include 3 example tickets per category in the prompt (few-shot in-context learning)
- Rationale: Tests whether showing the LLM actual ticket examples (from the training set) closes the gap with fine-tuned BERT. This connects to the REIC paper's finding that retrieval-augmented classification narrows the zero-shot performance gap[^3]

4.6 Dify Integration Plan

Once the best-performing model is identified, replace Dify's Question Classifier node:

- **If best model = Sentence-Transformer cosine router:** Encode KB category centroids offline, store as a lookup, call via a Dify Code node at inference
- **If best model = Fine-tuned BERT:** Host the model (HuggingFace Inference API, or Render/Modal) and call via an HTTP Request node in the Dify workflow
- **If best model = GPT few-shot:** Update the Dify Question Classifier node's category descriptions with example tickets from the training set — this requires no infrastructure change[22][21]

The key architectural point established in previous discussions applies here: routing categories must be **defined by KB structure**, but the classifier must be **trained on ticket language** — not derived from KB text alone.[26][5]

5. Results & Analysis (1–1.5 pages)

Present results in a comparison table:

Model	Features	Macro-F1	Weighted-F1	Accuracy	Notes
Naive Bayes	TF-IDF 1-2gram	—	—	—	Baseline
Logistic Regression	TF-IDF 1-2gram	—	—	—	Strong BoW baseline
Random Forest	TF-IDF 1-2gram	—	—	—	Tree ensemble
Semantic Router	Sentence-BERT	—	—	—	Embedding cosine
LR on Embeddings	Sentence-BERT	—	—	—	Learned boundary

Model	Features	Macro-F1	Weighted-F1	Accuracy	Notes
Fine-tuned BERT	Contextual	—	—	—	Expected best
GPT-4o-mini Zero-Shot	LLM prompt	—	—	—	Current system proxy
GPT-4o-mini Few-Shot	LLM + examples	—	—	—	Advanced LLM

Fill values from experiments. Expected ordering based on literature: Fine-tuned BERT ≥ Few-shot GPT > Semantic Router ≈ LR-on-embeddings > Logistic Regression-TF-IDF > Naive Bayes > Zero-shot GPT[5][6] [^3]

Error analysis: Report a confusion matrix for the best model. Identify which categories are most confused with each other — this typically reveals semantically overlapping categories in the KB that should be merged, or ambiguous tickets that require context beyond the query text alone.[^27]

Class imbalance effect: Compare per-class F1 scores between the smallest and largest categories. Minority classes will show notably lower F1 even for BERT — use this to argue for the relevance of SMOTE/weighted loss.[17][16]

6. Discussion & Conclusion (½–1 page)

Key arguments to make:

1. **Ticket-grounded training reliably outperforms zero-shot KB-based classification.** This confirms that Copilot's approach of generating classifiers from KB descriptions alone is fundamentally limited — it lacks exposure to the linguistic variation in real user queries.[5][3]
2. **Fine-tuned BERT captures semantic ambiguity that BoW models miss.** Examples: "my container hasn't moved" could be tracking, damage claim, or port dispute depending on context. Contextual representations handle this; TF-IDF cannot.[27][23]
3. **Embedding-based routing is a practical middle ground.** If computational constraints prevent BERT fine-tuning in production, sentence-transformer semantic routing offers near-BERT accuracy with near-zero latency and no retraining.[^5]
4. **Limitations:** The dataset is from a single company domain; results may not generalize. Ticket labels (if manually applied) introduce annotation subjectivity. Class imbalance remains a challenge for rare categories.
5. **Future work:** Multi-turn context (using conversation history, not just the last message); OOS (out-of-scope) detection for queries the KB cannot answer; continuous re-training as new ticket categories emerge.[28][29] [^27]

Part 2: Step-by-Step Technical Implementation

Step 1 — Label Your Tickets

The ticket dataset does not have ground-truth KB category labels — you need to create them.

Approach A (Recommended for the project):

1. Use sentence-transformers to embed all ticket texts
2. Cluster embeddings with K-Means (K = number of KB sections)
3. Manually review cluster centroids and assign KB category labels
4. Use Active Learning: train a preliminary classifier, identify low-confidence predictions, hand-label those tickets first[^8]
5. Aim for at least 50–200 labeled examples per category

Approach B (Faster, less rigorous):
Use GPT-4o to label tickets zero-shot, then spot-check 10–15% manually for quality. Acknowledge this as a limitation in the paper (synthetic/LLM-generated labels introduce noise).

Step 2 — Exploratory Data Analysis

Before building any model:

- Plot class distribution histogram → quantify imbalance ratio
- Calculate average and median ticket length (tokens)
- Build a word cloud per category to verify labels make sense
- Check for duplicate or near-duplicate tickets
- Flag tickets in a language other than English (Maersk is global — Danish, Chinese, and Spanish queries may appear)

Step 3 — Build the Baseline Pipeline (BoW + Classical ML)

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report

# TF-IDF + Naive Bayes
pipe_nb = Pipeline([
    ('tfidf', TfidfVectorizer(ngram_range=(1, 2), max_features=10000)),
    ('clf', MultinomialNB())
])
```

```
pipe_nb.fit(X_train, y_train)
print(classification_report(y_test, pipe_nb.predict(X_test),
      target_names=category_names))
```

Run the same structure for Logistic Regression and Random Forest. Report macro-F1 for each.

Step 4 — Sentence-Transformer Semantic Router

```
from sentence_transformers import SentenceTransformer
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

# Encode training tickets and compute per-category centroids
category_embeddings = {}
for cat in unique_categories:
    cat_texts = [t for t, l in zip(X_train, y_train) if l == cat]
    category_embeddings[cat] = model.encode(cat_texts).mean(axis=0)

# Route test tickets
def route(query):
    q_emb = model.encode([query])
    sims = {cat: cosine_similarity(q_emb, emb.reshape(1, -1))
            for cat, emb in category_embeddings.items()}
    return max(sims, key=sims.get)
```

This mirrors the approach benchmarked in Maksymenko et al. and is directly deployable in Dify via a Code node.[^5]

Step 5 — Fine-tune BERT

Use HuggingFace `transformers` with the Trainer API:

```
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer)
from torch.nn import CrossEntropyLoss
import torch

model_name = "bert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=len(unique_categories))

# Tokenize
def tokenize(batch):
    return tokenizer(batch['text'], truncation=True, padding='max_length',
                     max_length=128)

# Weighted loss for class imbalance
class_weights = compute_class_weight('balanced',
```

```
                                        classes=np.unique(y_train), y=y_train)
weights = torch.tensor(class_weights, dtype=torch.float)

training_args = TrainingArguments(
    output_dir='./bert-maersk',
    num_train_epochs=5,
    per_device_train_batch_size=16,
    learning_rate=2e-5,
    warmup_ratio=0.1,
    evaluation_strategy='epoch',
    save_strategy='best',
    load_best_model_at_end=True,
    metric_for_best_model='eval_macro_f1'
)
```

Fine-tuning BERT for 3–5 epochs with this setup mirrors the approach validated in both the BERT ticket classification literature and ModernBERT fine-tuning guides.[25][23]

Step 6 — GPT Zero-Shot and Few-Shot Baselines

```
import openai

CATEGORIES = {
    "shipment_tracking": "Questions about where a container or shipment is...",
    "booking_amendment": "Questions about modifying an existing booking...",
    # etc.
}

def classify_zero_shot(ticket_text):
    response = openai.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{
            "role": "system",
            "content": f"Classify the following customer query into exactly one categ
                       f"Categories: {json.dumps(CATEGORIES)}. Return ONLY the categc
        }, {
            "role": "user", "content": ticket_text
        }]
    )
    return response.choices.message.content.strip()
```

This reproduces what Copilot built for the current Dify classifier — zero-shot classification using KB category descriptions. Your empirical results will show exactly how much this underperforms ticket-trained models.[3][5]

Step 7 — Evaluate and Compare

```
from sklearn.metrics import f1_score, accuracy_score, confusion_matrix

def evaluate(y_true, y_pred, model_name):
    print(f"\n=== {model_name} ===")
    print(f"Macro-F1:    {f1_score(y_true, y_pred, average='macro'):.4f}")
    print(f"Weighted-F1: {f1_score(y_true, y_pred, average='weighted'):.4f}")
    print(f"Accuracy:    {accuracy_score(y_true, y_pred):.4f}")
    print(classification_report(y_true, y_pred,
                                target_names=category_names, digits=3))
```

Report macro-F1 as primary metric given class imbalance. Plot a confusion matrix heatmap for the best model using `seaborn.heatmap`.[14][13]

Step 8 — Deploy the Winning Model in Dify

Once the best classifier is identified, replace Dify's Question Classifier node:

Option A — Updated Zero-Shot Prompt (Cheapest):
In Dify's Question Classifier node, update each category description to include 5–10 example tickets from your training data. This is zero-code and immediately deployable.[22][21]

Option B — Embedding Router via Code Node:
Export the sentence-transformer centroids. Add a Dify **Code Node** (Python) before the Question Classifier that computes cosine similarity and passes the predicted category as a variable downstream.[^21]

Option C — External BERT API via HTTP Node:
Host your fine-tuned BERT model (Render, Modal, or HuggingFace Inference API). Add a Dify **HTTP Request Node** that calls your classifier API and routes based on the response. This is the highest-accuracy option and the production-ready approach.[30][22]

Part 3: Key Framing for the Exam

The examiner will probe your understanding of *why* each method works differently. Prepare the following arguments:

Why does BoW fail on shipping tickets?
TF-IDF treats words as independent features — "my shipment is delayed" and "goods have not arrived" have zero lexical overlap but the same intent. N-gram models partially help but still cannot capture cross-sentence semantic equivalence.[31][19]

Why does zero-shot GPT underperform fine-tuned BERT here?
GPT has general world knowledge but no exposure to Maersk's specific terminology, booking workflows, or the distribution of query types. Fine-tuned BERT adapts its attention heads to the specific domain vocabulary and the statistical patterns of how Maersk

customers phrase requests. As REIC shows, this gap is approximately 25–35 macro-F1 points.[23][3]

Why is macro-F1 the right metric?
With imbalanced classes, accuracy is dominated by the majority class (e.g., shipment tracking). Macro-F1 gives equal weight to every category — including rare ones like dangerous goods inquiries that are arguably the *most* important to route correctly.[15][13]

Why route on KB structure but train on tickets?
The KB defines *what can be answered*; tickets define *how users actually ask*. Routing categories must come from the KB (you can only route to content that exists), but the classifier must be trained on ticket language to recognize those categories in natural user phrasing. Copilot's error was using only KB descriptions without any ticket signal.[3][5]

References

1. POWERED CHATBOTS AND LOGISTICS PRICING AT MAERSK ... - AI-powered technology emerges as a solution to streamline these operations. Brief Introduction of th...
2. Maersk: AI-Driven Strategy Yields 20% Operational Efficiency Increase - The integration of AI and unified data systems led to around a 20% increase in operational efficienc...
3. [2506.00210] REIC: RAG-Enhanced Intent Classification at Scale - In this paper, we introduce REIC, a Retrieval-augmented generation Enhanced Intent Classification ap...
4. REIC: RAG-Enhanced Intent Classification at Scale - ACL Anthology - In this paper, we introduce REIC, a Retrieval-augmented generation Enhanced Intent Classification ap...
5. [PDF] Benchmarking Conversation Routing in Chatbot Systems Based on ... - The results show best practices for building a semantic router layer of chat application and provide...
6. Improving Automated Categorization of Customer Requests with ... - In this paper, we focus on the categorization of tickets in service desk systems. We employ modern n...
7. improving automated categorization of customer requests ...
8. [PDF] Enhancing Chatbot Intent Classification using Active Learning ... - Enhancing Chatbot Intent Classification using Active Learning Pipeline for. Optimized Data Preparati...
9. Enhancing Chatbot Intent Classification using Active Learning Pipeline for Optimized Data Preparation
10. [PDF] Customer Support Ticket Categorization and Prioritization Using ... - This paper presents an Natural Language Processing (NLP) - driven system designed to automate the ca...
11. [PDF] Machine Learning for Intent Classification in an Educational Chatbot - educational chatbot is built using different NLP techniques as well as different. ML algorithms. A c...
12. [PDF] multi-label intent classification for educational chatbot: a ... - In this study, since the data analyses the user's intent in the form of text data from a chatbot, te...
13. Micro, Macro & Weighted Averages of F1 Score, Clearly Explained - This article looks at the meaning of these averages, how to calculate them, and which one to choose ...

14. A Closer Look at Classification Evaluation Metrics and a Critical ... - iii) All metrics except Accuracy and weighted F1 show chance baseline correction. Strict chance base...
15. [PDF] Comparing Micro, Macro, and Weighted F1-Scores - bnaic 2025
16. Synthetic minority oversampling technique - Wikipedia
17. A comprehensive evaluation of
18. Naive Bayes, Logistic Regression, & Fine-Tuned BERT - YouTube - Text Classification powers spam filters, news categorization, and sentiment analysis! This complete ...
19. [PDF] Features extraction based on Naive Bayes algorithm and TF-IDF for ... - In this study, we explore the use of BERT for news classification, comparing its performance with tr...
20. Automated Research Article Classification and Recommendation ... - Our experiments show that Logistic Regression with TF–IDF consistently yields the best classificatio...
21. Question Classifier - Dify Docs - The Question Classifier node intelligently categorizes user input to route conversations down differ...
22. Question Classifier | Dify - It classifies the user's input question, directing it to different downstream knowledge base queries...
23. Automated Customer Ticket Classification using BERT - Inspirisys - Optimize Customer Support Efficiency with BERT-Powered Ticket Classification! Explore the Power of A...
24. unionai-oss/bert-llm-classification-pipeline: fine tune model ... - GitHub - The tutorial notebook will guide you through the steps to fine-tune a BERT-LLM model on a classifica...
25. Fine-tune classifier with ModernBERT in 2025 - Philschmid - 1. Setup environment and install libraries · 2. Load and prepare the classification dataset · 3. Fin...
26. Ticket routing - Claude API Docs - Ticket routing is a type of classification task. Claude analyzes the content of a support ticket and...
27. Enhancing Customer Service Chatbots with Context-Aware NLU ... - In this paper, we have introduced a context-aware NLU model that incorporates both, the customer que...
28. Generating Hard-Negative Out-of-Scope Data with ChatGPT for ... - We present an automated technique to generate hard-negative OOS data using ChatGPT. We use our techn...
29. Improved Out-of-Scope Intent Classification with Dual Encoding and ... - In this paper, we introduce Dual Encoder for Threshold-Based Re-Classification (DETER), a method for...
30. Dify Workflow – Phase 2 | Question Classifier: Intent Routing - ... workflow starts making decisions. This lesson focuses on: What the Question Classifier does (and...
31. Intent Classification: Techniques for NLP Models - Intent classification assigns user input to a specific intent label; core to chatbots, virtual assis...